Module Builder

From fixed capabilities to an open extension platform

The direction

Hardware agent + device setup + workflow blocks

The first proof

Remove an existing capability. Download it. Restore it.

Build locally. Add capabilities. Share optionally.

Adding capabilities currently means changing the app

Hardware, device setup, and workflows are defined across compiled frontend and backend code.

Hardware support

Fixed capability branches for Camera, GPIO, Sensors, and MQTT.

Device setup

Templates, forms, wizard choices, and guides are compiled into the app.

Workflow blocks

Types, configuration, validation, presentation, and execution use fixed mappings.

Every new choice should arrive as an installable package.

One module can connect the whole experience

A package may contribute one feature or a complete device-to-workflow experience.

Hardware adapter

Read a sensor, operate a device, or expose events.

Device setup

Add a card, configuration fields, connection test, and guide.

Workflow blocks

Add actions, data operations, triggers, and starter workflows.

Definitions, executable behavior, guides, and examples travel together.

The direction is agreed; delivery details remain open

Keep the product ambition separate from unresolved operating choices.

Agreed direction

Full host access, including the hardware agent.

Dynamic hardware, Devices, and workflow contributions.

Removable default packages and a real local demo library.

Local authoring; optional central publication.

Still to define

Runtime tooling and package boundaries.

Supported OS and architecture matrix.

Public publishing, review, and update policies.

Owners, cost, staffing, and delivery estimates.

Local freedom is agreed. Public-library policy still needs design.

Established platforms offer useful patterns

Use proven extension patterns while keeping Edge Studio's local and hardware focus.

Node-RED

Packages extend the node palette with behavior, editor resources, and examples.

Lesson: package the complete capability.

Home Assistant

Integration manifests describe devices; separate apps declare runtime and host needs.

Lesson: distinguish device semantics from host operations.

VS Code

Manifests declare contributions; extension hosts execute code.

Lesson: stable contracts plus clear publisher trust.

Borrow contracts and lifecycle ideas; preserve our workflow semantics.

Full host access means trusting host software

A module may run privileged code, install dependencies, and extend the hardware agent.

Freedom

Custom drivers, languages, setup logic, services, and hardware operations.

Responsibility

The operator trusts the module and its dependencies with the host and its data.

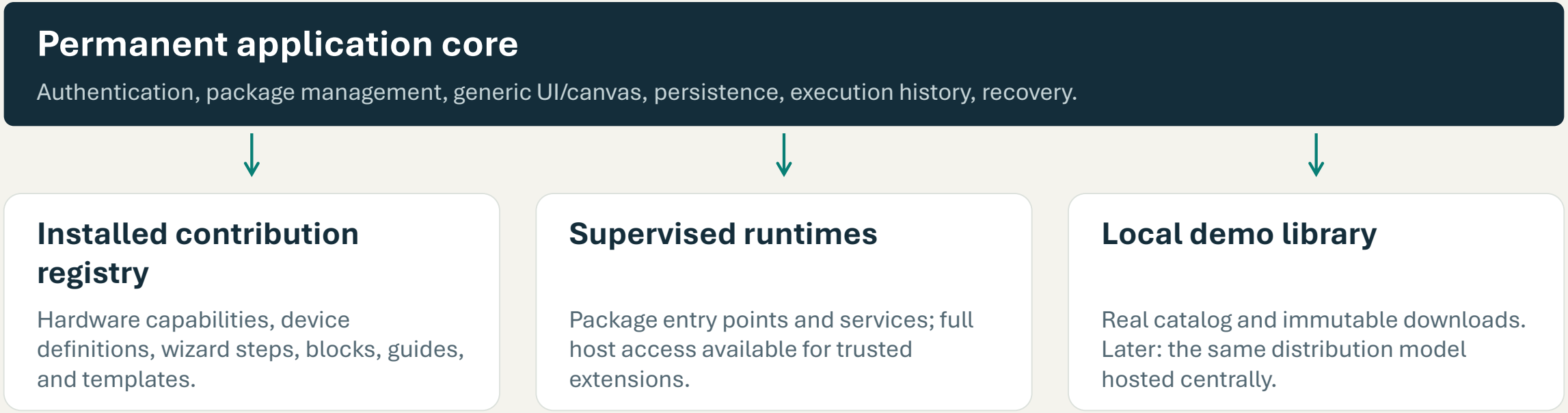
Honest controls

Show identity and access; manage activation, logs, backups, and recovery.

Process separation improves operations; it does not constrain root-capable code.

Keep a small core that can restore every extension

Dynamic contributions enter through versioned contracts and an extension manager.



The app must remain recoverable with zero extensions installed.

Default capabilities become ordinary packages

Existing implementations become the first extension library and reference examples.

Today

Hard-coded choices inside hardware, device, and automation features.

Extract

Give each capability an owner, contract, version, configuration, and lifecycle.

Default extensions

Camera, GPIO, Sensors, MQTT, HTTP, webhook, system data, and workflow contributions.

No hidden built-in fallback after a capability is uninstalled.

M1: prove one complete lifecycle before scaling

Start with HTTP JSON Source device setup and its Fetch block contribution.

01

Baseline

Configure the device and run a workflow.

02

Remove

Uninstall; show missing device and block states.

03

Download

Retrieve real package bytes from the separate local library.

04

Activate

Install and activate the compatible release.

05

Verify

Restore behavior, preserve history, and survive app restart.

Expand only after the original configuration and behavior are restored.

M2: migrate all defaults and verify real hardware

Repeat the proven lifecycle across hardware, device setup, and workflow capabilities.

Dynamic hardware

Package Camera, GPIO, I2C Sensors, and Local MQTT, including setup and repair.

Dynamic experience

Load device forms/guides and block configuration/behavior from installed packages.

Complete restoration

Remove all extensions, retain a usable core, then reinstall the default set.

A working HTTP demo is the start; hardware parity is a separate gate.

M3: prove that developers can extend the app

Begin tooling once the first contracts and lifecycle work; overlap with later migration.

Create

Start from templates, or import source and supported runtimes.

Develop

Edit code/schemas; preview setup and blocks; inspect tests and logs.

Release

Package an immutable version; activate locally or export it.

Share

Install on another instance; optionally publish when the public service exists.

A new module must work on a second instance without core-source edits.

The library starts locally and grows into a service

Separate the lifecycle demo from operating a public extension ecosystem.

Local demo / in plan

Separate catalog service with real downloads.
Versioned default packages and compatible installs.
Offline local activation and execution.
No public accounts or moderation required.

Public service / later

Publisher accounts and public package pages.
Central package/image storage and distribution.
Signing, checks, reporting, and advisories.
Operational ownership, policy, and funding.

Central hosting distributes modules; local execution remains independent.

Compare the practical implementation choices

Keep host freedom while choosing maintainable contracts and packaging.

In-process plugins

Benefit: direct calls.
Cost: shared failures.
Position: avoid as default.

Supervised host runtimes

Benefit: host freedom.
Cost: dependency ownership.
Position: first host path.

Container runtimes

Benefit: bundled dependencies.
Cost: ARM/device setup.
Position: packaging option.

Start with supervised runtimes and generated UI; retain extension hooks.

Prove extensibility without losing existing users' work

Preserve configured instances and observable behavior through removal and reinstall.

Preserve

Keep IDs, device settings, secrets, workflows, and execution history.

Explain

Render missing-package placeholders with clear restoration steps.

Recover

Exercise restart, interrupted installs, failed setup, version conflicts, and backups.

Uninstall removes a capability; data deletion is a separate action.

Delivery gates control scope and investment

Each gate produces evidence that supports the next delivery decision.

M1 / Lifecycle proof

Inventory + contracts + HTTP slice + local download/reinstall.

M2 / Default parity

All capability families packaged; physical hardware and restoration verified.

M3 / Authoring proof

New block and device built in-app and installed on a second instance.

M4 / Public service

Future phase: publishing, central operations, trust, governance, and support.

Estimate broader delivery after contracts and M1 expose the real coupling.

Ecosystem maintenance is part of the product

An open extension system creates continuing responsibilities beyond the editor.

Platform ownership

Contracts, compatibility, core recovery, documentation, and default packages.

Hardware support

Supported OS/CPU/runtime matrix and physical validation for official packages.

Public operations

Hosting, publisher support, key management, moderation, and incident response.

Keep the first gate narrow; identify owners before public publishing.

Align on the next gate and the operating questions

Keep the approved product direction; make the next delivery decision concrete.

Next delivery discussion

Assign the M1 owner and agree on its acceptance demonstration.

Choose the initial supported test environment.

Review M1 evidence before broad capability migration.

Later policy discussion

Who maintains official and community packages?

What publishing/review and source policy fits the public library?

Who funds operations, and what update/withdrawal policy preserves operator control?

Proceed through a bounded lifecycle proof, then review evidence.

The proof is a capability that returns intact

Remove it. Download it. Activate it. Keep the user's work.

First evidence

An existing device and block restored from a real local library.

Platform evidence

Every default capability installable; new modules created without core edits.

Then use the same platform to build something new.

Appendix: what to borrow, and where the analogy stops

Similar projects solve parts of this problem; none remove our integration work.

Node-RED

Borrow: a package with runtime, editor help, and examples.

Difference: our library owns central artifact delivery.

Home Assistant

Borrow: distinct integration metadata and host-runtime configuration.

Difference: our supervisor is new work.

VS Code

Borrow: contribution contracts and an extension host.

Difference: our root-capable modules carry broader host trust.

The contracts are the reusable idea; our lifecycle still needs proof.

Appendix: public-library policy choices

Each choice balances developer access, operator control, and service responsibility.

Publication

Immediate listing: fast access, greater moderation exposure.

Review first: slower access, more operating work.

Updates

Manual activation: operator timing, slower patch uptake.

Automatic activation: faster fixes, more disruption risk.

Withdrawal

Advisories: operator control, installed risk remains.

Remote disable: faster response, can interrupt real devices.

Publication rules can be stricter than local module execution.

Appendix: official research sources

External facts are sourced; Edge Studio recommendations are our interpretation.

Node-RED / custom nodes and packaging

https://nodered.org/docs/creating-nodes/
https://nodered.org/docs/creating-nodes/packaging

Home Assistant / integrations and apps

https://contributors.home-assistant.io/core/integrations/manifest/
https://contributors.home-assistant.io/apps/reference/configuration/

VS Code / contributions and extension execution

https://code.visualstudio.com/api/references/extension-manifest
https://code.visualstudio.com/api/advanced-topics/extension-host
https://code.visualstudio.com/docs/configure/extensions/extension-runtime-security

Detailed citations and alternatives are in the companion discussion document.

Appendix: trust and internal references

The plan defines delivery; the discussion records unresolved choices and alternatives.

TUF / update metadata and repository trust

https://theupdateframework.io/docs/overview/

Docker / daemon control and host exposure

https://docs.docker.com/engine/security/

Repository documents

Implementation: docs/plans/module-builder.md
Discussion: docs/plans/module-builder/discussion.md
Presentation source: this stakeholder brief

Current application security policy changes only when implementation lands.